Obfuscapk를 통한 난독화 진행

목차

Contents

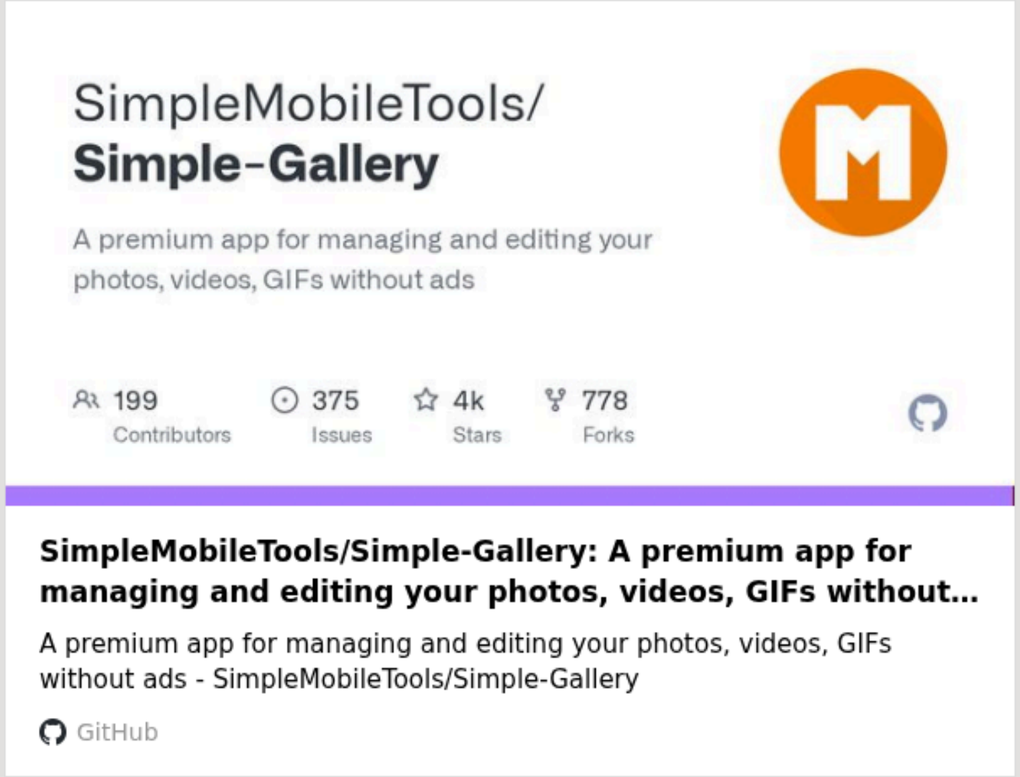
01 난독화 실패한 apk 파일 분석

02 실패한 난독화 기법

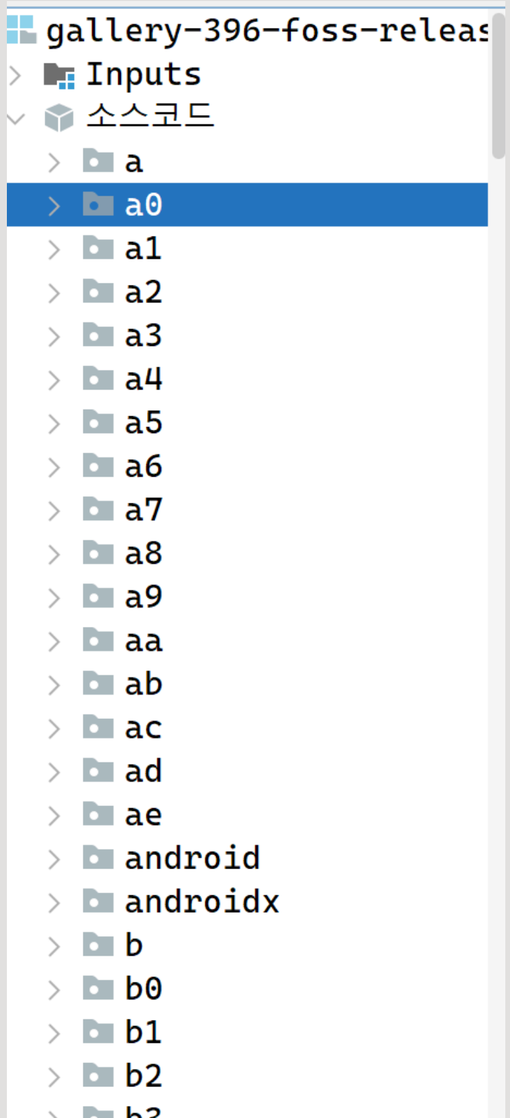
03 난독화 실패 원인 분석

04 해당 난독화를 다른 apk에 적용

Simple-Gallery 앱 난독화 진행



apk 원본 파일 분석



이와 같은 구조로 소스 폴더 하위의
폴더들이 구성이 되어있습니다.
이는 이미 해당 APK 파일이 난독화가
되어있는 상태라는 것을 알 수 있습니다.
상세한 이유는 다음과 같습니다.

1. 의미 없는 이름

```
a
  a(ColorSpace$Rgb$TransferParameters) double
  b() ColorSpace$Named
  c(Bitmap) ColorSpace
  d(View) void
  e(View, String[]) void
b
  f313a View
  f314b s
  f315c AutofillManager
  b(View, s) void
c
d
  f316a d
  {...} void
  d() void
  a(ViewStructure, int) int
  b(ViewStructure, int) ViewStructure
  c(ViewStructure, int, int, int, int, int, int)
  d(ViewStructure, int, String, String, String) v
e
f
  a(ColorSpace$Rgb$TransferParameters) double
  b() ColorSpace$Named
  c(String, FileAttribute[]) Path
  d(View, Collection, int) void
```

1. 의미 없는 이름

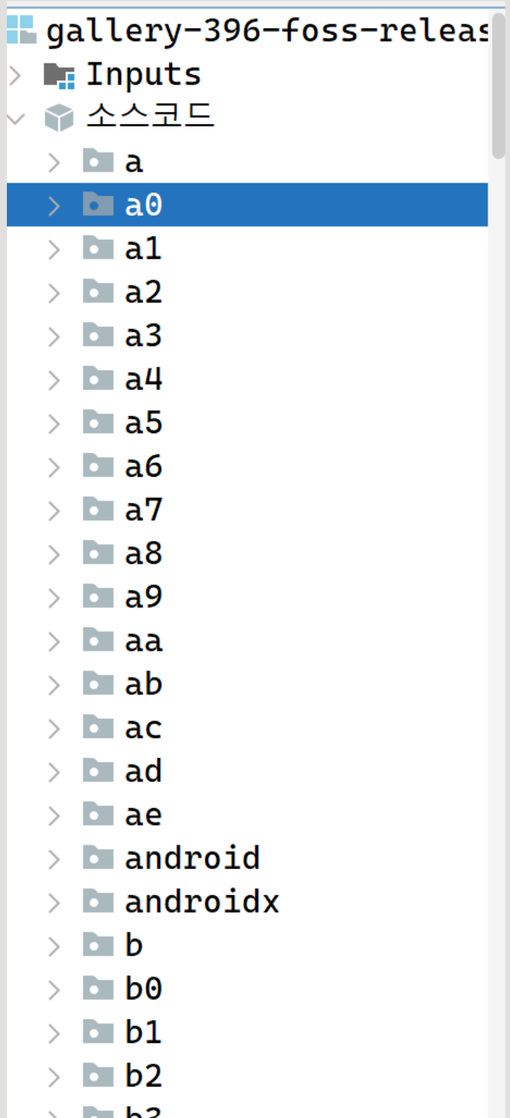
```java
package a1;

import android.view.ViewStructure;

/* loaded from: classes.dex */
public final class d {

    /* renamed from: a, reason: collision with root package name */
    public static final d f316a = new d();

    public final int a(ViewStructure viewStructure, int i10) {
        return viewStructure.addChildCount(i10);
    }

    public final ViewStructure b(ViewStructure viewStructure, int i10) {
        return viewStructure.newChild(i10);
    }

    public final void c(ViewStructure viewStructure, int i10, int i11, int i12, int i13, int i14,
        viewStructure.setDimens(i10, i11, i12, i13, i14, i15);
    }

    public final void d(ViewStructure viewStructure, int i10, String str, String str2, String str3
        viewStructure.setId(i10, str, str2, str3);
    }
}
```

1. 의미 없는 이름

- 클래스 이름:
  public final class b0(원래는 UserProfile이나 MainActivity 처럼 의미있는 이름이었을 것입니다.)
- 패키지 이름:
  a0와 같은 의미없는 이름
- 필드(변수)이름:
  f149b와 같은 이름
- 매개변수 이름:
  i10 과 같은 이름

원래의 기능을 유추할 수 없도록 의미없는 이름으로 바뀌었습니다

2.의미 없는 패키지 구조



위 사진 처럼 폴더 목록은 앱의 소스코드 패키지 구조를 보여줍니다.
이 역시 패키지 구조를 난독화 한 결과입니다.
(android,androix 와 같은 폴더는 안드로이드 시스템 라이브러리이므로 난독화에서 제외되었습니다)

3.AndroidManifest.xml 분석

이렇게 난독화가 진행이 되었으면 AndroidManifest.xml의
Android:name도 이에 맞게 변경되지 않았을까? => NO

```
        android:targetSdkVersion="33"/>
    <uses-permission
        android:name="android.permission.WRITE_EXTERNAL_STORAGE"
        android:maxSdkVersion="40"/>
    <uses-permission android:name="android.permission.SET_WALLPAPER"/>
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
    <uses-permission android:name="android.permission.ACCESS_MEDIA_LOCATION"/>
    <uses-permission android:name="android.permission.READ_MEDIA_IMAGES"/>
    <uses-permission android:name="android.permission.READ_MEDIA_VIDEO"/>
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
    <uses-permission android:name="android.permission.MANAGE_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.MANAGE_MEDIA"/>
    <queries>
        <package android:name="com.simplemobiletools.gallery.debug"/>
        <package android:name="com.simplemobiletools.gallery"/>
        <package android:name="com.simplemobiletools.thankyou.debug"/>
        <package android:name="com.simplemobiletools.thankyou"/>
        <intent>
            <action android:name="android.intent.action.GET_CONTENT"/>
            <category android:name="android.intent.category.OPENABLE"/>
            <data android:mimeType="*/*"/>
        </intent>
```

3.AndroidManifest.xml 분석

AnroidManeifest.xml에는 명확한 이름의 클래스들이 적혀있는 것을 볼 수 있습니다.

3.AndroidManifest.xml 분석

- 어떻게 이것이 가능 할까?

아마 이 apk 파일은 안드로이드 난독화 도구 ProGuard 또는 R8을 통해 keep 규칙을 활용해서 난독화를 진행했을 것입니다

- 왜 AndroidManifest의 이름은 그대로 두었을 까?

안드로이드 시스템은 AndroidManifest.xml에 적힌 정확한 클래스 이름을 보고 앱의 특정 화면(Activity)를 실행시키거나 특정기능(Service,Reciver)을 호출합니다.
만약 com.simplemobiletools.gallery.pro.activities.MainActivity라는 이름을 a.b.c로 바꿔버리면, 안드로이드 시스템은 MainActivity를 찾지 못해 앱이 시작조차 되지 않고 충돌하게 됩니다.

이러한 “외부에서 호출되는 진입점”들은 반드시 원래 이름을 유지해야 합니다.

3.AndroidManifest.xml 분석

현재 상황 분석

1. AndroidManifest.xml 의 이름들이 멀쩡한 이유 난독화 규칙에 따라 AndroidManifest.xml에 등록된 모든 Activity, Service, Receiver 등의 이름은 바뀌지 않도록 예외 처리(-keep)된 것입니다. 이는 앱이 정상적으로 작동하기 위한 필수 조치입니다.
2. 내부 코드와 폴더의 이름이 변경된 이유

- d.java나 a0, a1 같은 클래스와 패키지들은 Manifest에 직접 등록되지 않은, 앱 내부에서만 사용되는 코드들입니다.
- 이들은 외부에서 직접 호출될 일이 없기 때문에, 난독화 도구가 마음껏 짧고 의미 없는 이름으로 바꿔버린 것입니다. 이것이 바로 앱의 핵심 로직을 숨기는 난독화의 주된 목적입니다.

3.AndroidManifest.xml 분석

결론

이 APK 파일은 난독화가 이미 적용되어 있는 상태이고, AndroidManifest.xml의 이름은 의도적으로 설정된 정상적인 난독화의 결과입니다.
즉, 이는 외부와의 약속(진입점 이름) 은 지키면서, 내부의 중요한 로직은 효과적으로 숨기는 매우 표준적이고 올바른 난독화 방식입니다.

해당 apk 파일로 난독화 진행

RandomManifest 난독화 진행

1. 문제 발생(난독화 실패)

해당 APK 파일로 RANDOMMANIFEST 난독화를 진행하였습니다.
하지만 계속해서 난독화가 실패하여 APK 파일 생성에 실패하였습니다.

2. 문제 원인 파악

RandomManifest 난독화 때문이거나, original.apk 파일 자체가 Obfuscapk 내부의 빌드 도구(apktool)와 호환되지 않을 가능성이 존재하여
문제의 원인을 정확히 파악하기 위해, 실제 난독화 과정을 제외하고 순수하게 '재포장'만 테스트해 보았습니다.

RandomManifest 같은 실제 난독화 기술은 적용하지 않고, 단순히 APK를 분해했다가 다시 Rebuild ,NewAlignment,NewSignature
만 수행하여 제대로 조립되는지 확인하는 테스트입니다.

=> apk 파일이 잘 만들어 지는 것을 확인했습니다.

3. 문제 원인 분석

- Obfuscapk의 시도: AndroidManifest.xml 파일에 가짜 액티비티나 서비스 같은 임의의 데이터를 추가하려고 합니다.
- 실패 가능성:
  - 구조적 복잡성: 이미 복잡하게 구성된 Manifest 파일에 새로운 구성요소를 안전하게 추가하는 것이 기술적으로 실패할 수 있습니다.
  - 우선순위 문제: 다른 규칙들과 충돌하면서 Manifest를 수정하는 작업이 정상적으로 완료되지 않았을 수 있습니다.

4. 다음 조치

- 오류 없이 repackaged.apk 파일이 생성되어, original.apk 파일 자체나 빌드 과정에는 문제가 없는 것을 확인했습니다. 이 경우, RandomManifest 난독화 기법이 이 APK와 충돌하는 것일 수 있습니다.
- 해결책: RandomManifest 대신 다른 난독화 기법을 사용하였습니다.

=>ClassRename 난독화 진행

ClassRename 난독화 진행

1. 문제 발생(난독화 실패)

로그를 보면, Rebuild (재조립) 단계에서 실패하고 있습니다.

2. 문제 원인 파악

- Obfuscapk의 시도: com.simplemobiletools.gallery.pro.activities.MainActivity의 이름을 a.b.c 등으로 바꾸려고 합니다.
- 기존 APK의 저항: 하지만 이 APK는 이미 MainActivity의 이름을 보존하라는 강력한 keep 규칙을 내장하고 있을 확률이 99%입니다. 이 규칙은 앱이 시스템에 의해 실행되기 위해 반드시 필요합니다.
- 결과: Obfuscapk의 Rename 명령이 기존의 더 중요한 keep 규칙에 의해 무시되거나 거부됩니다. 따라서 Manifest에 등록된 주요 컴포넌트들의 이름은 바뀌지 않고 그대로 남게 됩니다. (물론, keep 규칙으로 보호받지 않는 다른 내부 클래스들은 이름이 바뀌었을 수도 있습니다.)

앞선 두 난독화 실패 이유 분석

Simple Mobile Gallery Pro와 같은 정식 출시된 앱은 빌드될 때 이미 “proguard-rules.pro”라는 파일을 통해 자체적인 코드 최적화 및 난독화 규칙을 가지고 있습니다. Obfuscapk는 이미 빌드된 APK 파일을 분해해서 그 위에 추가적인 난독화를 시도하는 도구입니다. 이 과정에서 기존 APK가 가지고 있던 -keep 규칙과 Obfuscapk의 기능이 충돌할 수 있습니다.

다른 apk 파일에서는 RandomManifest 난독화가 제대로 진행될까?

Simple-Calender 앱 난독화 진행

SimpleMobileTools/
Simple-Calendar

A simple calendar with events, tasks, customizable colors, widgets and no ads.

202 Contributors | 296 Issues | 4k Stars | 1k Forks

Releases · SimpleMobileTools/Simple-Calendar

A simple calendar with events, tasks, customizable colors, widgets and no ads. - SimpleMobileTools/Simple-Calendar

GitHub

RandomMaifest 난독화 진행

성공적으로 apk 파일 생성

obfuscation_working_dir	2025-09-25 오후 4:25	파일 폴더	
calendar.apk	2025-09-25 오후 4:20	APK 파일	10,539KB
RandomManifest.apk	2025-09-25 오후 4:39	APK 파일	10,844KB
RandomManifest.apk.idsig	2025-09-25 오후 4:39	IDSIG 파일	94KB

결과 분석

jadx를 통해 두 apk 파일의 AndroidManifest.xml 파일을 얻고,
VSCode의 비교기를 통해서 두 AndroidManifest.xml 파일 비교를 진행하였습니다.

Random_AndroidManifest.xml ↔ original_AndroidManifest.xml U

2주차-난독화 결과 확인 > apk_test2 > original_AndroidManifest.xml

C:\Users\82102\Obfuscapk\2주차-난독화 결과 확인 • 강조 표시한 항목 포함

```xml
<manifest xmlns:obfuscation="http://schemas.android.com/apk/res/android"
    obfuscation:versionCode="240"
    obfuscation:versionName="6.23.1"
    obfuscation:installLocation="0"
    obfuscation:compileSdkVersion="34"
    obfuscation:compileSdkVersionCodename="14"
    package="com.simplemobiletools.calendar.pro"
    platformBuildVersionCode="34"
    platformBuildVersionName="14">
    <uses-sdk
        obfuscation:minSdkVersion="23"
        obfuscation:targetSdkVersion="34"/>
    <uses-permission obfuscation:name="android.permission.POST_NOTIFICATIONS"/>
    <uses-permission
        obfuscation:name="android.permission.WRITE_EXTERNAL_STORAGE"
        obfuscation:maxSdkVersion="28"/>
    <uses-permission obfuscation:name="android.permission.WAKE_LOCK"/>
    <uses-permission obfuscation:name="android.permission.USE_EXACT_ALARM"/>
    <permission
        obfuscation:name="com.simplemobiletools.calendar.pro.DYNAMIC_RECEIVER_NOT_EXPORTED_PER
        obfuscation:protectionLevel="0x2"/>
    <queries>
        <package obfuscation:name="com.simplemobiletools.thankyou"/>
        <package obfuscation:name="com.simplemobiletools.contacts.pro.debug"/>
        <package obfuscation:name="com.simplemobiletools.thankyou.debug"/>
        <package obfuscation:name="com.simplemobiletools.contacts.pro"/>
```

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="240"
    android:versionName="6.23.1"
    android:installLocation="auto"
    android:compileSdkVersion="34"
    android:compileSdkVersionCodename="14"
    package="com.simplemobiletools.calendar.pro"
    platformBuildVersionCode="34"
    platformBuildVersionName="14">
    <uses-sdk
        android:minSdkVersion="23"
        android:targetSdkVersion="34"/>
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
    <uses-permission android:name="android.permission.READ_CONTACTS"/>
    <uses-permission android:name="android.permission.VIBRATE"/>
    <uses-permission android:name="android.permission.READ_CALENDAR"/>
    <uses-permission android:name="android.permission.WRITE_CALENDAR"/>
    <uses-permission android:name="android.permission.WAKE_LOCK"/>
    <uses-permission android:name="android.permission.USE_EXACT_ALARM"/>
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
    <uses-permission
        android:name="android.permission.SCHEDULE_EXACT_ALARM"
        android:maxSdkVersion="32"/>
    <uses-permission
        android:name="android.permission.WRITE_EXTERNAL_STORAGE"
        android:maxSdkVersion="28"/>
    <queries>
        <package android:name="com.simplemobiletools.contacts.pro.debug"/>
        <package android:name="com.simplemobiletools.contacts.pro"/>
        <package android:name="com.simplemobiletools.thankyou.debug"/>
        <package android:name="com.simplemobiletools.thankyou"/>
    </queries>
    <queries>
        <package android:name="com.simplemobiletools.calendar.debug"/>
        <package android:name="com.simplemobiletools.calendar"/>
```

결과 분석

AndroidManifest.xml 파일의 태그 위치가 Random 하게 변하고 쓰레기 삽입이 들어가 있는 것을 VSCode의 비교를 통해 확인했습니다

RandomManifest 난독화 기술 효과

-원리
실제로 존재하지 않는 가짜 정보를 넣어서 분석가를 교란 시키는 것
실제 앱의 기능은 건드리지 않으면서, AndroidManifest.xml 파일에만 무작위로 생성된 가짜 컴포넌트들을 대량으로 삽입하는 것입니다.

-목표

1. 자동화된 분석 도구 무력화: 정적 분석 도구들은 AndroidManifest.xml에 등록된 정보를 100% 믿고 분석을 시작합니다. 여기에 가짜 정보가 섞여 있으면 도구는 존재하지 않는 코드를 찾으려다가 오류를 발생시키거나 분석을 멈추게 됩니다
2. 분석가의 혼란 유도: 사람 분석가는 수많은 컴포넌트 목록을 보며 어떤 것이 앱의 진짜 기능이고 어떤 것이 가짜 '쓰레기' 정보인지 구분해야합니다/ 이 과정에서 분석의 초점을 흐리고, 귀중한 시간을 낭비하게 만듭니다.

RandomManifest 난독화 기술 효과

존재하지 않는 가짜 컴포넌트 정보를 대량으로 추가해서

1. 분석 시간 증가: 분석가는 수십, 수백 개의 컴포넌트 목록을 보며 어떤 것이 진짜이고 가짜인지 구별하는 작업이 필요합니다. 이 과정에서 시간과 노력을 낭비하게 됩니다.
2. 자동화 도구 오류 유발: 정적 분석 도구는 Manifest에 선언된 컴포넌트를 기반으로 코드의 구조를 파악합니다, 가짜 컴포넌트 정보 때문에 도구가 존재하지 않는 코드를 찾으려다가 오류가 발생할 수 있습니다.

RandomManifest 난독화 기술 효과

결론적으로 앱 설계도인 AndroidManifest.xml에 가짜 쓰레기 정보를 섞어 설계도 자체를 신뢰할 수 없게 만드는 아주 효과적인 교란 기법입니다.

THANK YOU

감사합니다.